CUDA Ray Tracing Project Report

By Artur Quarra & Caleb Brooks

5 / 3 / 2026

Table of Contents

Introduction

This project implements a recursive ray tracing system in Python, supporting both CPU execution and GPU acceleration via CUDA with Numba. The renderer generates realistic three-dimensional scenes by simulating the behavior of light rays as they interact with objects in a virtual environment. The completed system supports recursive reflection, recursive refraction, Phong-style lighting, multiple camera viewpoints, and GPU-accelerated rendering for improved performance.

The rendered environment contains several spheres positioned above a checkerboard plane and illuminated by multiple light sources. Different material properties assigned to each sphere demonstrate reflective and refractive behavior. Multiple camera configurations render the same scene from different perspectives, allowing evaluation of both the rendering pipeline and the camera model.

The primary objective of the project is to explore the mathematical and computational foundations of ray tracing while investigating the performance advantages of GPU parallelization. The final implementation demonstrates the practical application of computer graphics concepts, including geometric intersection testing, lighting models, recursive ray generation, and camera transformations.

Objectives

The project focuses on several major rendering and graphics requirements. The first objective is to implement recursive ray tracing capable of rendering realistic lighting interactions between rays and scene geometry. Another important objective is to add plane intersection support alongside sphere intersection testing, allowing the scene to contain both curved and flat surfaces.

The implementation also includes reflective surfaces that recursively spawn secondary rays to simulate mirror-like behavior. Refraction simulates transparent or glass-like materials according to Snell's Law. Multiple camera viewpoints render the same environment from several perspectives. Finally, the renderer uses Numba and CUDA to accelerate rendering via GPU parallel processing.

Scene Description

The final rendered scene contains three spheres positioned above a checkerboard plane. Each sphere uses different material parameters to showcase distinct visual effects, including diffuse shading, reflection, and refraction. The checkerboard plane provides a reference surface that makes shadows, reflections, and perspective easier to observe. Two separate light sources illuminate the scene. The lighting arrangement emphasizes reflections and specular highlights on the objects' surfaces. Multiple predefined camera viewpoints appear in the implementation, including front, left-side, right-side, wide-angle, and close-up perspectives. The arrangement of the scene demonstrates depth relationships, occlusion, reflective interactions, and refractive distortion. The varying camera positions further confirm that the camera transformation system and projection calculations operate correctly across different viewpoints.

Mathematical Background

Ray Equation

Ray tracing relies on the ray equation, which describes a parametric line traveling through three-dimensional space. A ray follows the equation:

$$r(t) = e + td, \quad t \geq 0$$

where e represents the ray origin and d represents the direction vector. The parameter t determines the position along the ray.

Sphere Intersection

Sphere intersection testing determines whether a ray collides with a sphere in the scene. A sphere centered at c with radius r satisfies the equation:

$$|p - c|^2 = r^2$$

Substituting the ray equation into the sphere equation produces a quadratic equation whose solutions determine whether an intersection occurs. The discriminant of the quadratic equation determines whether the ray misses the sphere, touches the sphere tangentially, or intersects the sphere at two points. The renderer selects the smallest positive solution as the visible intersection point.

Plane Intersection

An infinite plane through a point and a normal vector defines the checkerboard floor. The renderer computes the ray-plane intersection parameter using:

$$t = \frac{n \cdot (p_0 - e)}{n \cdot d}$$

where n represents the plane normal, p_0 represents a point on the plane, e represents the ray origin, and d represents the ray direction. If the denominator approaches zero, the renderer treats the ray as parallel to the plane and rejects the intersection.

Surface Normals

The renderer requires surface normals for lighting calculations. For spheres, the renderer computes the normal vector from the sphere center to the intersection point. Plane normals remain constant across the surface of the plane. The renderer uses these normals in diffuse, specular, reflection, and refraction calculations.

Diffuse Lighting

The renderer implements diffuse shading using Lambertian reflection. The diffuse intensity depends on the angle between the surface normal and the light direction:

$$I_d = k_d \max(0, n \cdot l)$$

This model creates realistic matte shading, with surfaces facing the light source appearing brighter.

Specular Lighting

The renderer implements specular highlights using a Phong lighting model with a half-vector formulation. The specular component creates shiny highlights that simulate the appearance of polished or reflective materials.

Reflection

The renderer generates reflection rays recursively using the reflection equation:

$$r = d - 2(d \cdot n)n$$

where d represents the incoming ray direction, and n represents the surface normal. Recursive reflection allows surfaces to reflect other objects within the scene.

Refraction

The renderer simulates transparent materials using Snell's Law:

$$n_1 \sin\theta_1 = n_2 \sin\theta_2$$

The refracted ray direction depends on the incident angle, the surface normal, and the indices of refraction of the two materials. The renderer discards invalid refracted rays when total internal reflection occurs.

Fresnel Approximation

The renderer uses Schlick's approximation to blend reflection and refraction more realistically:

$$F(\theta) = F_0 + (1 - F_0)(1 - \cos\theta)^5$$

This approximation improves realism by increasing the intensity of reflections at grazing angles.

Recursive Ray Tracing

The recursive ray tracing process begins by generating a primary viewing ray from the camera into the scene. The renderer identifies the nearest intersection point, computes local shading, and, when appropriate, recursively spawns reflected and refracted rays. The renderer accumulates the resulting color contributions until the recursion reaches a predefined depth. Recursive tracing improves realism by allowing surfaces to interact visually through reflections and transparency.

Camera Model

The renderer uses a standard camera coordinate system based on orthonormal basis vectors. The implementation computes the camera orientation from the eye position, look-at position, and up vector. These vectors define the viewing direction and projection plane for generated rays. The implementation supports multiple camera configurations, enabling the renderer to capture the scene from different viewpoints while maintaining proper perspective projection. Rendered outputs from several viewing angles verify that the camera system correctly transforms and projects the scene.

Tone Mapping and Gamma Correction

The renderer converts high dynamic range lighting values into displayable image intensities using ACES tone mapping followed by gamma correction. Tone mapping compresses extremely bright lighting values into a visible range while preserving visual detail. Gamma correction adjusts the image for proper display on standard monitors. These post-processing steps improve the visual realism and brightness balance of the rendered outputs.

Implementation

The project includes both a CPU-based renderer and a CUDA-accelerated GPU renderer implemented with Numba. The CPU version provides a fallback rendering mode and simplifies debugging, while the GPU version significantly improves rendering speed through parallel execution.

The renderer supports several rendering modes, including preview rendering, CPU rendering, GPU rendering, and high-resolution 4K GPU rendering. The code automatically generates multiple viewpoints, allowing the same scene to render from different camera positions without modifying the scene geometry. CUDA acceleration requires adapting many computations into GPU-compatible operations. The implementation optimizes ray generation, intersection testing, lighting calculations, and recursive effects to function efficiently within the CUDA execution model.

Results

The final rendered outputs successfully demonstrate recursive reflection, recursive refraction, multiple camera viewpoints, checkerboard reflections, and GPU-accelerated rendering. Reflective surfaces accurately mirror the surrounding geometry, while refractive materials realistically distort the environment. The pink sphere clearly reflects the checkerboard floor and nearby objects. The center sphere behaves either as a reflective mirror or as a transparent, glass-like material, depending on the assigned parameters. The rear sphere adds depth and occlusion, enhancing the scene's spatial realism. The checkerboard plane enhances the visibility of reflections, shadows, and perspective effects. The multiple rendered viewpoints confirm that the camera system correctly transforms and projects the scene from different angles. GPU rendering substantially reduces render times compared to CPU execution, particularly for higher-resolution outputs and deeper recursion levels.

Challenges

Several technical challenges arose during development. One major challenge was producing brighter, more visually realistic output while maintaining stable recursive lighting behavior. Another challenge involved optimizing performance to reduce CPU rendering time. CUDA configuration and dependency management also created difficulties. Missing CUDA libraries and compatibility issues required troubleshooting before successful GPU execution became possible. Additional challenges included maintaining numerical stability in reflection and refraction calculations and tuning material properties so the final rendered images appeared visually convincing. Managing recursive ray depth while preventing high computational cost also required careful tuning. Balancing rendering quality and performance remained an important consideration throughout development.

Limitations

The current renderer primarily supports spheres and planes, which limits the geometric complexity of renderable scenes. The renderer also simplifies soft shadow rendering and approximates glossy reflections only partially. Scene setup currently depends on manually defined parameters within the program. Although GPU acceleration significantly improves performance, recursive ray tracing still requires substantial computational resources at high resolutions and recursion depths.

Future Work

Several improvements could extend the functionality and realism of the renderer. Future development could add support for triangle meshes and complex polygonal geometry. Bounding Volume Hierarchies (BVH) or other spatial acceleration structures could also improve intersection performance.

Additional rendering improvements could include physically accurate soft shadows, advanced glossy reflection models, texture mapping, and global illumination techniques. Real-time scene editing tools and an interactive graphical interface could further improve usability. More advanced GPU optimization strategies could also increase rendering efficiency for complex scenes and higher image resolutions.

Conclusion

This project successfully demonstrates the implementation of a recursive ray-tracing system in Python, accelerated with CUDA via Numba. The renderer supports sphere and plane intersections, recursive reflections, recursive refractions, multiple camera viewpoints, and GPU-accelerated rendering. The completed implementation demonstrates both the mathematical foundations and practical programming techniques required for modern ray tracing systems. The final rendered outputs clearly showcase reflections, refractions, shading, perspective projection, and recursive lighting interactions from multiple viewpoints. The project also highlights the importance of GPU acceleration in computationally intensive graphics applications. Overall, the final system provides a strong demonstration of core computer graphics concepts and establishes a solid foundation for future improvements and extensions to rendering.